

Tards TCG 游戏引擎架构文档

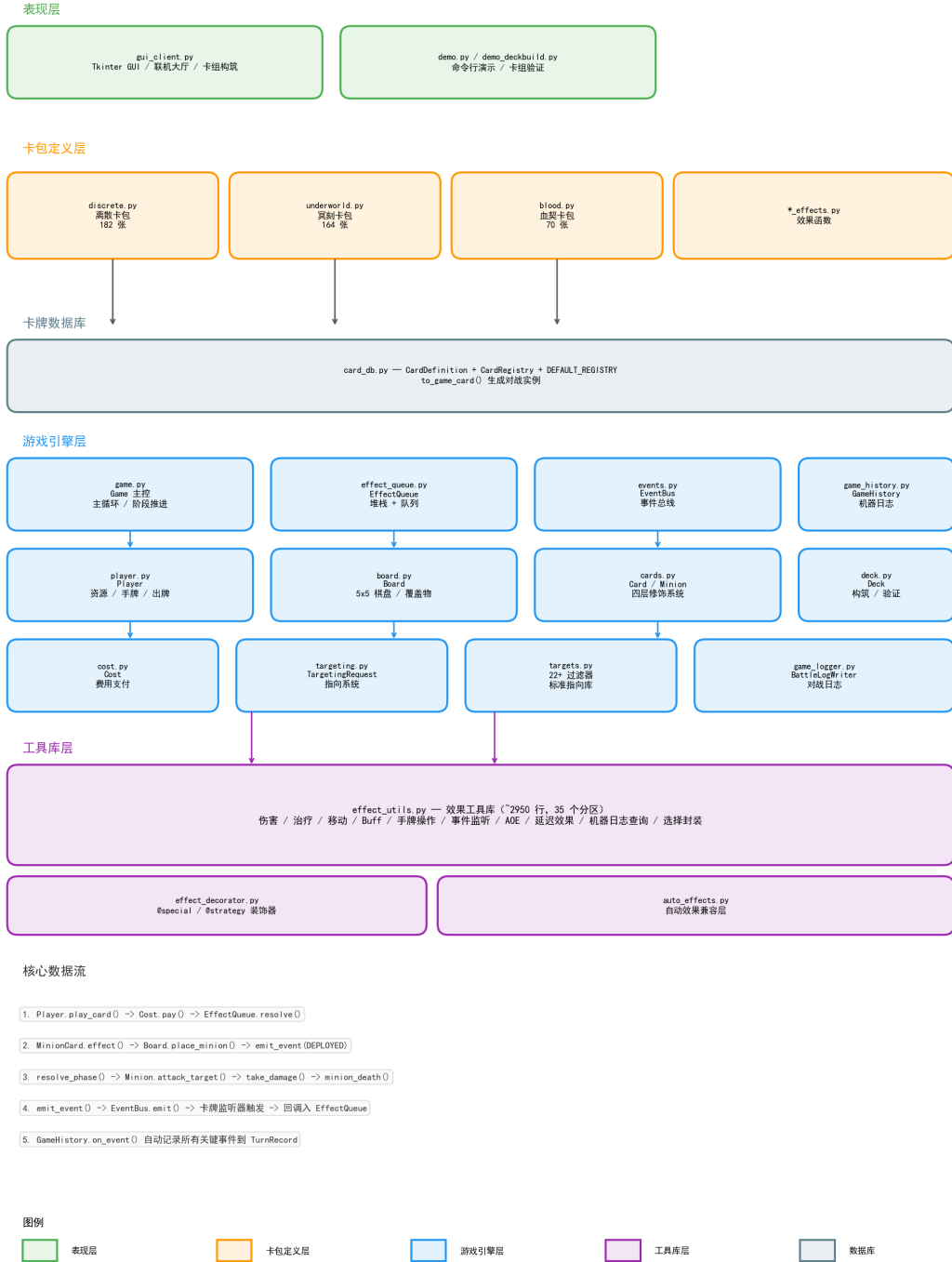
模块分层、核心类关系与数据流

生成日期: 2026-05-06

一、模块架构总图

Tards TCG 引擎架构图

模块分层与核心依赖关系



Tards TCG 游戏引擎架构文档

> 本文档描述 Tards 卡牌游戏引擎的模块分层、核心类关系、数据流与事件系统。

> 撰写时间：2026-05-06

一、项目概述

Tards 是一款原创集换式卡牌游戏（TCG），采用 Python 3.14 开发。引擎设计遵循以下原则：

- 事件驱动：所有状态变更通过 EventBus 发射事件，卡牌效果通过监听事件触发。
- 堆栈系统：主动效果（出牌、攻击）进入 LIFO 堆栈，连锁效果（亡语等）进入 FIFO 队列。
- 禁止硬编码：核心引擎不引用任何具体卡牌名称，卡牌行为完全通过 effect_fn / special_fn / 事件监听器实现。
- 工具库优先：通用战斗、伤害、移动、复制等行为封装在 effect_utils.py 中，卡牌效果文件保持精简。

二、模块分层

表现层 (Presentation)		
└── gui_client.py	Tkinter GUI 客户端（本地/联机对战、卡组构筑器）	
└── demo.py	命令行演示入口（随机 AI 快速测试）	
└── demo_deckbuild.py	卡组构筑演示与验证	
卡包定义层 (Card Pools)		
└── card_pools/discrete.py	离散卡包注册区（182 张）	
└── card_pools/underworld.py	冥刻卡包注册区（164 张）	
└── card_pools/blood.py	血契卡包注册区（70 张）	
└── card_pools/*_effects.py	各卡包效果函数	
└── card_pools/effect_decorator.py	@special / @strategy 装饰器	
└── card_pools/effect_utils.py	效果工具库（~2900 行，核心 API）	
游戏引擎层 (Core Engine)		
└── tards/game.py	游戏主控（主循环、阶段推进、全局状态）	
└── tards/player.py	玩家实体（资源、手牌、出牌流程）	
└── tards/board.py	棋盘状态（5×5 格子、异象位置、覆盖物）	
└── tards/cards.py	卡牌基类与战场异象实体（MinionCard / Minion）	
└── tards/effect_queue.py	效果队列与堆栈系统	
└── tards/events.py	事件总线（EventBus + GameEvent）	
└── tards/cost.py	费用系统（Cost + can_afford/pay）	
└── tards/targeting.py	通用指向系统（TargetingRequest + TargetPicker）	
└── tards/targets.py	标准指向过滤器库（~22 个函数）	
└── tards/auto_effects.py	自动效果兼容层（转发到 effect_utils）	
└── tards/game_history.py	机器日志模块（GameHistory + TurnRecord）	
└── tards/game_logger.py	对战日志记录（BattleLogWriter）	
└── tards/card_db.py	卡牌注册数据库（CardDefinition + CardRegistry）	
└── tards/deck.py	卡组构建与验证	
└── tards/constants.py	事件常量与通用关键词	

三、核心引擎架构图



3.1 Game（游戏主控）

Game 是整局对局的唯一主控对象，负责：

- 生命周期管理：start_game() → 抽牌阶段 → 出牌阶段 → 结算阶段 → 回合循环
- 阶段推进：draw_phase() / action_phase() / resolve_phase()
- 事件发射：emit_event() —— 所有状态变更的统一入口
- 效果调度：通过 EffectQueue 管理堆栈和队列
- 全局状态：延迟效果、部署限制、伤害替换、指向保护、雕像拼装队列
- 历史记录：GameHistory 自动追踪每回合的关键变量

3.2 Player（玩家实体）

Player 管理玩家侧的所有状态：

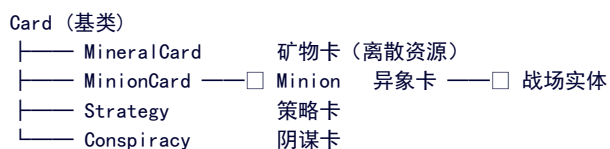
- 资源系统：t_point / t_point_max / c_point / c_point_max / b_point / s_point
- 牌堆管理：card_deck（牌库）/ card_hand（手牌）/ card_dis（弃牌堆）
- 出牌流程：card_can_play()（合法性检查）→ play_card()（费用支付 → EffectQueue 结算）
- 沉浸度增益：immersion_points（离散/冥刻/血契）
- 状态标志：_skip_next_draw、_cost_modifiers、_deploy_buffs、_on_develop_callbacks

3.3 Board（棋盘）

5x5 格子系统，两行友方区域 + 两行敌方区域 + 中间中立行：

- minion_place: Dict[(r, c), Minion] —— 表面异象
- cell_underlay: Dict[(r, c), Minion] —— 底层覆盖物（藤蔓/漂浮物宿主）
- 部署合法性检查：is_valid_deploy() —— 处理水路、高地、河岸、独行、协同等限制
- 前排目标获取：get_front_minion() —— 处理潜水/潜行（结算阶段）、恐惧、空袭等

3.4 Cards（卡牌与异象实体）

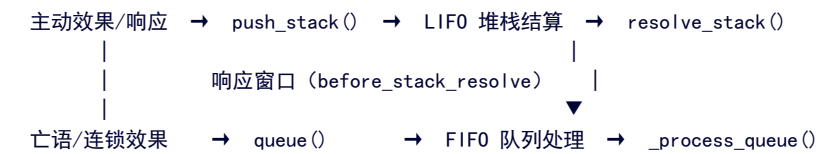


Minion（战场实体）的四层修饰系统：

层级	属性前缀	持久性	说明
-----	-----	-----	-----
base	base_	永久	初始面板与关键词
permanent	perm_	跨回合	增益/减益效果
temporary	temp_	本回合	回合结束自动清除
aura	_aura_	动态	由场上其他异象提供的光环

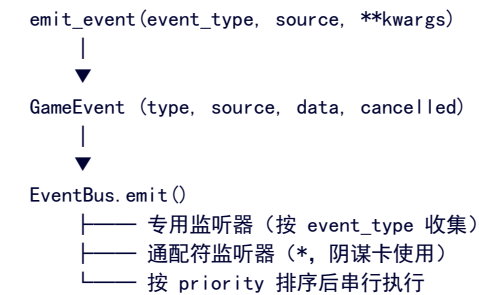
recalculate() 将四层修饰合并为最终面板：current_attack / current_health / keywords。

3.5 EffectQueue（效果队列与堆栈）



- resolve(): 主入口。若未在结算中，推入堆栈 → 结算堆栈 → 处理队列。
- 响应窗口：每帧结算前发射 EVENT_BEFORE_STACK_RESOLVE，允许推入新的响应效果。
- 最大深度：500（防止无限递归）。

3.6 EventBus（事件总线）



事件类型三层结构：

层级	示例	特性
-----	-----	-----
Before	EVENT_BEFORE_DAMAGE, EVENT_BEFORE_DEPLOY	可取消/可修改
主事件	EVENT_DAMAGED, EVENT_DEPLOYED	只读，已发生
After	EVENT_AFTER_DAMAGE, EVENT_AFTER_DEPLOY	只读，后置处理

四、游戏主循环数据流

4.1 对局生命周期

start_game()

```

├── 初始化玩家血量、资源、沉浸度开局增益
├── 抽取初始手牌（4张）
├── 执行 on_game_start 回调（如血渍怀表置入卡组顶）
├── while not game_over:
│   run_turn()
│       ├── draw_phase() [抽牌 + T/C 资源恢复]
│       ├── action_phase() [玩家交替出牌/拍铃/兑换]
│       └── resolve_phase() [异象按列攻击、结算伤害]

```

4.2 出牌阶段（action_phase）数据流

```

玩家行动 → Game.action_provider()
├──
├── "brake" → 拉闸，结束出牌阶段
├── "bell" → 拍铃，检查是否改变过 T 点
├── "exchange" → 兑换矿物/松鼠
├── "play" → Player.play_card(serial, target, game)
│   │
│   ▼
│   1. card_can_play() → 费用/目标合法性
│   2. Cost.pay() → 扣除资源（T/B/S/C/矿物）
│   3. _cards_played_this_phase += 1
│   4. 按卡牌类型分派：
│       ├── MinionCard → EffectQueue.resolve("部署", deploy_fn)
│       ├── Strategy → EffectQueue.resolve("打出策略", play_fn)
│       └── Conspiracy → 注册到 EventBus 通配符监听
│   5. emit_event(EVENT_CARD_PLAYED)
│   6. Game.history.on_event() 自动记录到机器日志

```

4.3 结算阶段（resolve_phase）数据流

```

resolve_phase()
├── 清空本回合临时状态（_deployed_this_turn、_damage_dealt_to_players_this_turn）
├── 按列倒序遍历（水路 4 → 高地 0）
│   ├── 收集本列可攻击异象
│   ├── 按先攻等级 → 距中线距离 → side 排序
│   └── while 还有攻击者:
│       ├── 同先攻组内一起攻击
│       ├── 处理视野预设目标 / 横扫 / 串击 / 穿刺
│       ├── 处理防空、冰冻翻倍、尖刺反弹
│       └── 伤害 → take_damage() → 检查死亡 → minion_death()
├── 回合结束：清理临时 buff、削减层数关键词、处理成长
└── emit_event(EVENT_TURN_END)

```

五、费用与资源系统

```

Cost
├── t: int    T 点（每回合自然增长，上限 8/10）
├── c: int    C 点（矿物兑换获得，上限回满）
├── b: int    B 点（鲜血，献祭异象获得，回合结束清空）
├── s: int    S 点（灵魂，特定效果/沉浸度获得）
├── ct: int   CT 点（优先扣 C，不足再扣 T）
└── minerals: Dict[str, int] 矿物需求

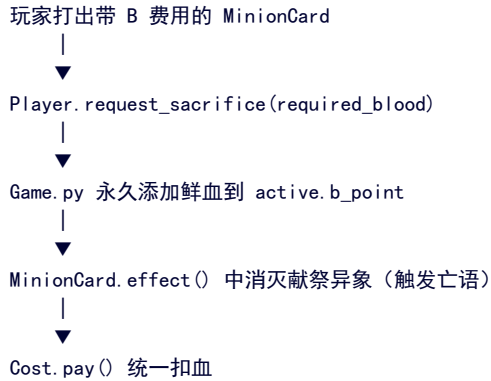
```

```

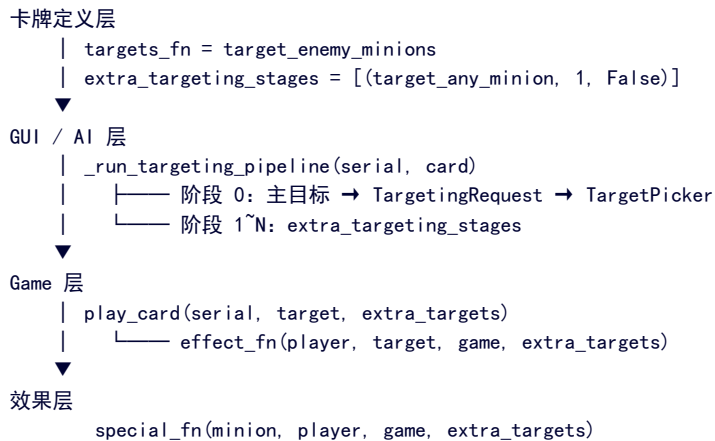
支付流程（Cost.pay(player)）：
1. 扣 T 点（t_point_change）
2. 扣 B/S（直接减）
3. CT 优先扣 C 再扣 T
4. 从手牌移除对应矿物卡

```

献祭机制 (Sacrifice) :



六、指向系统架构



TargetingRequest 数据对象:

```
@dataclass
class TargetingRequest:
    valid_targets: List[Any]      # 合法目标 (位置/异象/玩家/手牌卡)
    count: int = 1                # 需选择数量
    allow_repeat: bool = False    # 是否允许重复
    prompt: str                   # 提示文字
    on_confirm: Callable          # 确认回调
    on_cancel: Callable           # 取消回调
```

七、效果工具库分层

card_pools/effect_utils.py 是卡牌效果实现的核心依赖, 约 2900 行, 按功能分为 35 个分区:

- 效果工具库 (effect_utils.py)
- |—— 1. 附魔书开发卡池
 - |—— 2. 伤害与治疗 (deal_damage_to_minion / heal_player)
 - |—— 3. 场上异象操作 (summon_token / destroy_minion / return_minion_to_hand)
 - |—— 4. 移动框架 (move / swap / shift)
 - |—— 5. Buff / 沉默 / 关键词 (buff_minion / gain_keyword / silence_minion)
 - |—— 6. 手牌与牌库操作 (draw_cards / copy_card_to_hand / discover_from_deck_top)
 - |—— 7. 异象查询 (all_enemy_minions / has_tag / get_adjacent_positions)
 - |—— 8. 资源管理 (gain_resource / lose_resource)

- 9. 亡语与抽取 (add_deathrattle / set_draw_trigger)
- 10. 事件监听器 (on() 体系 / add_event_listener)
- 11~12. 兼容层事件包装
- 13. 开发机制辅助 (deploy_card_copy / auto_attack)
- 14. 延迟效果 (delay_to_next_turn / delay_to_turn_end)
- 15. 状态追踪框架 (track_stat / track_per_turn / track_event_per_turn)
- 16. 即时对战 (initiate_combat)
- 17. AOE / 批量效果 (damage_all_enemies / freeze_enemies_in_columns)
- 18. 高级目标选择器 (weakest_enemy_minion / nearest_enemy_minion)
- 19. "如可能"条件执行 (if_possible_then / if_resource_then)
- 20. 临时效果管理 (give_temp_keyword_until_turn_end)
- 21. 牌库操作增强 (reveal_top_of_deck / draw_cards_of_type)
- 22. 伤害重定向与免疫 (redirect_damage)
- 23. 全局地形覆盖 (register_terrain_enforcement)
- 24. 机器日志查询 API (GameHistory 封装, ~40 个函数)
- 25. 选择接口封装 (request_choice_index / request_target / request_targets)
- 26. 场上异象遍历 (all_minions_on_board, 带去重)
- 27. MinionCard 复制 (copy_minion_card)
- 28~29. 延迟效果 / 猪灵掉落物
- 30. 选择接口封装
- 31. 场上异象遍历
- 32. MinionCard 复制
- 33. 延迟效果封装
- 34. 目标合法性快捷检查
- 35. 机器日志高级查询

八、日志模块架构

8.1 分层设计

文件日志层	tards/game_logger.py → BattleLogWriter 输出到 logs/battle_YYYYMMDD_HHMMSS.log	
全局状态日志 (跨对局持久)	Game._state_log (List[Dict]) 记录: minion_death, health_lost, t_max_lost	
机器日志 (新模块)	GameHistory (Game._history) TurnRecord × N + 当前快照 自动由 emit_event 驱动更新	
回合级计数器 (旧机制, 兼容)	Game._deployed_this_turn Game._damage_dealt_to_players_this_turn Game._strategies_played_this_turn	
实体级快照	Minion._last_damage_source/_type/_amount	

8.2 GameHistory 数据结构

```

GameHistory
—— _records: List[TurnRecord]      # 已归档回合
—— _current: TurnRecord            # 当前回合快照

TurnRecord (单回合)
—— 出牌统计: cards_played[Player], minions_deployed[Player],
            strategies_played[Player], total_strategies_played,
            minerals_played[Player], conspiracies_activated[Player]
—— 资源费用: sacrifices_made[Player], blood_spent[Player],
            developed_count[Player]
—— 抽弃磨: cards_drawn[Player], cards_discarded[Player],

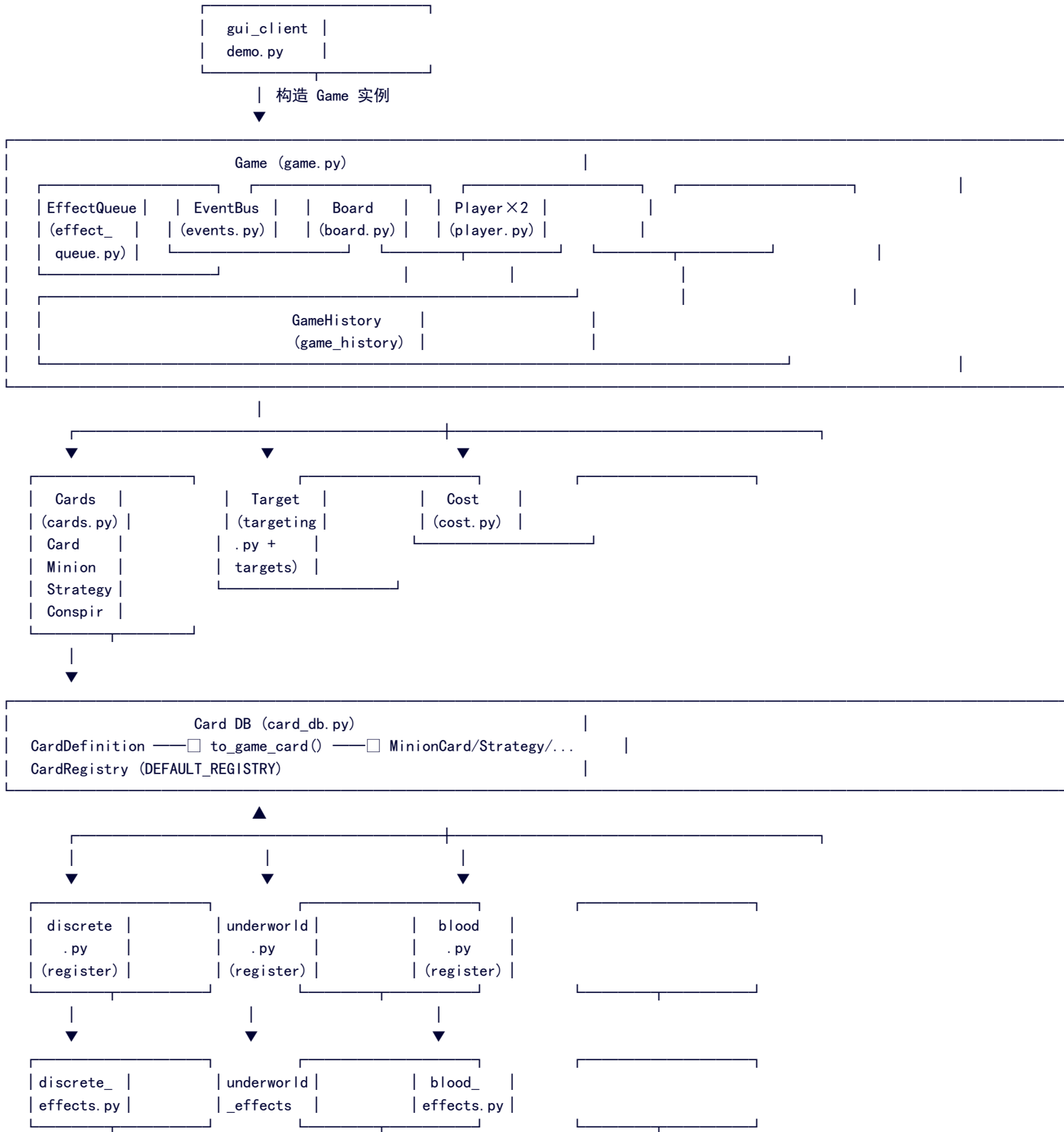
```

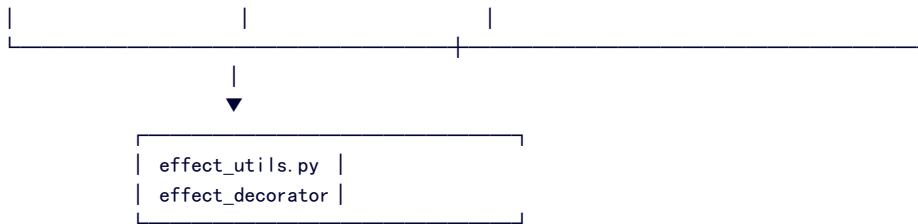
```

| cards_milled[Player]
| 战斗生存: damage_dealt_to_players[Player],
|           damage_dealt_to_minions[Player],
|           healing_received[Player], attacks_made[Player]
| 资源上限: t_max_lost[Player]
| 阶段级: health_lost_this_phase[Player]
| 实体列表: deployed_minions[], died_minions[], sacrificed_minions[]

```

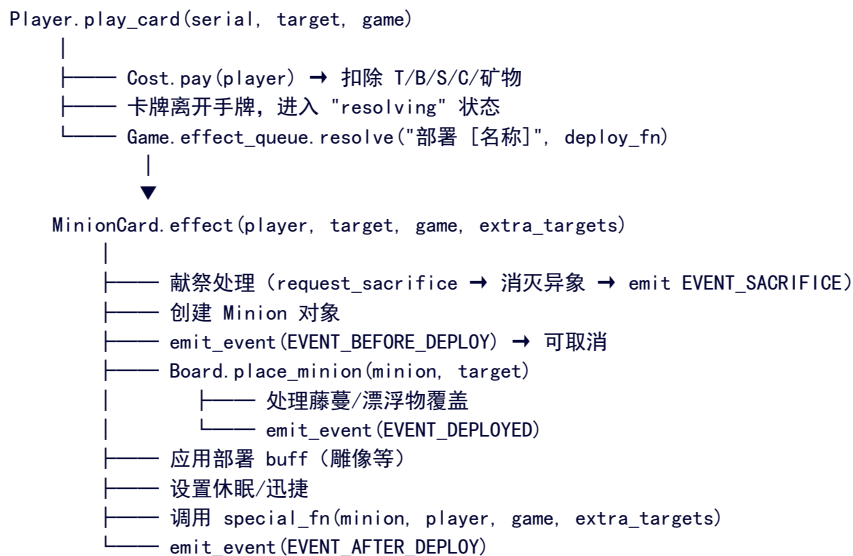
九、模块依赖关系总图



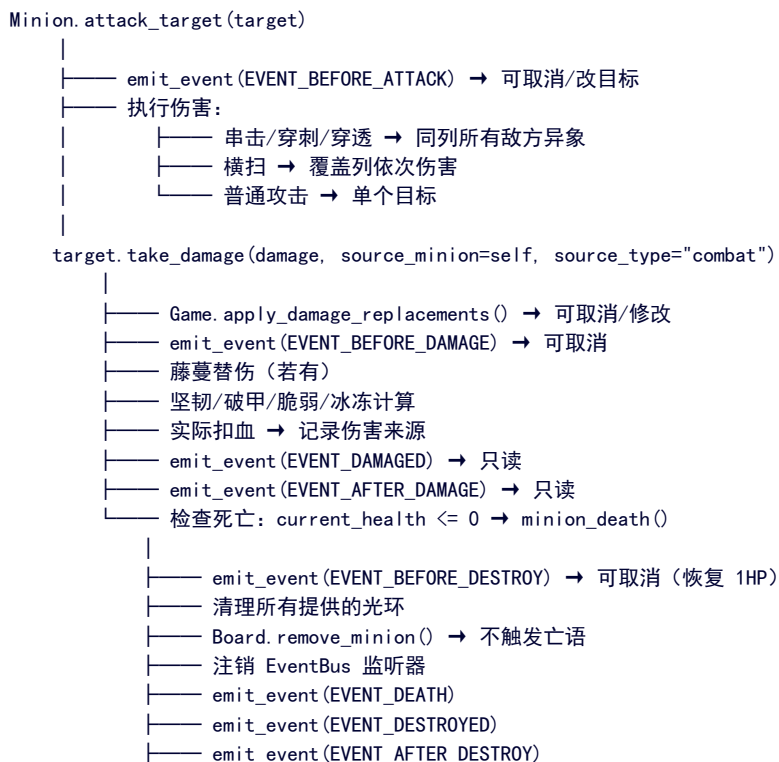


十、关键数据流示例：卡牌打出 → 部署 → 战斗 → 死亡

10.1 异象部署 (MinionCard)



10.2 战斗伤害 (Minion.attack_target)



十一、扩展指南

11.1 添加新卡牌效果

1. 在 `card_pools/xxx_effects.py` 中编写 `effect_fn` / `special_fn` / `condition_fn`。
2. 优先使用 `effect_utils.py` 中的已有 API，避免手写底层逻辑。
3. 在 `card_pools/xxx.py` 中调用 `register_card()` 注册。
4. 如需新机制，扩展 `effect_utils.py` 或 `game.py` 中的通用回调/事件。

11.2 添加新事件类型

1. 在 `tards/constants.py` 中定义常量。
2. 在需要发射的地方调用 `game.emit_event(EVENT_XXX, ...)`。
3. 如需自动记录到机器日志，在 `GameHistory.on_event()` 中添加处理逻辑。

11.3 添加新指向过滤器

1. 在 `tards/targets.py` 中编写函数，签名统一为 `(player, board) -> List[Any]`。
2. 在 `card_pools/xxx.py` 的 `register_card()` 中作为 `targets_fn` 传入。

附录：文件清单与职责速查

文件	行数	核心职责
-----	-----	-----
<code>tards/game.py</code>	~1280	游戏主循环、阶段推进、堆栈/事件调度
<code>tards/player.py</code>	~500	玩家资源、手牌管理、出牌流程
<code>tards/board.py</code>	~600	5×5 棋盘、异象位置、部署合法性
<code>tards/cards.py</code>	~940	卡牌基类、Minion 实体、四层修饰系统
<code>tards/effect_queue.py</code>	~200	LIFO 堆栈 + FIFO 队列 + 响应窗口
<code>tards/events.py</code>	~150	EventBus、GameEvent、优先级排序
<code>tards/cost.py</code>	~200	Cost 解析、 <code>can_afford</code> 、 <code>pay</code>
<code>tards/targeting.py</code>	~100	TargetingRequest、TargetPicker
<code>tards/targets.py</code>	~180	22+ 标准指向过滤器
<code>tards/game_history.py</code>	~450	TurnRecord、GameHistory、自动事件追踪
<code>tards/card_db.py</code>	~400	CardDefinition、CardRegistry、 <code>register_card</code>
<code>tards/deck.py</code>	~300	卡组构建、验证规则
<code>card_pools/effect_utils.py</code>	~2950	效果工具库（35 个分区）
<code>card_pools/effect_decorator.py</code>	~80	@special / @strategy 装饰器
<code>gui_client.py</code>	~2200	Tkinter GUI、本地/联机对战、卡组构筑

demo.py	~150	命令行演示入口
---------	------	---------